

STUDENT OS

Product Requirements Document (PRD)

File: 01_PRD.md

Document purpose: Master product definition for AI-assisted development of the Student OS application.

Version: 1.0

Status: Development Specification

1. Product Overview

Student OS is a student-focused productivity and academic management platform that brings a student's daily academic information, tasks, timetable, attendance, exams, notes, notifications and progress into one clear workflow. The product should answer the practical question: **“What is happening today, what is pending, and what should I do next?”**

The product is not intended to replace an institution's official ERP or academic system. The initial version should work with manual, mock, imported or authorized integration data. Any future college-system integration must use an officially permitted API or data source; the application must not scrape, bypass authentication, or imitate unauthorized access.

2. Problem Statement

- Students often receive academic information across different places and have difficulty seeing the complete picture.
- Timetable, assignments, exams, attendance, events and personal study tasks are usually treated as separate things.
- Students may know an assignment exists but still miss its deadline because there is no connected reminder workflow.
- Attendance percentages alone do not clearly tell students how many classes they can or cannot miss.
- Exam dates are visible, but preparation planning is often disconnected from the actual exam schedule.
- Students need a simple daily command center rather than a collection of unrelated dashboards.

3. Product Vision

Create a modern “Student Operating System” that converts academic information into clear daily actions, planning, reminders and progress insights while remaining simple enough for everyday student use.

4. Target Users

- Primary: college/university students.
- Secondary: students managing assignments, projects, exams and personal study schedules.
- Future: teachers, mentors, institutions and authorized administrators, if the product expands into multi-role workflows.

5. Product Goals

- Give students a single place to understand their academic day.
- Reduce missed classes, assignments, exams and reminders.
- Connect timetable, tasks, calendar and notifications.
- Make attendance understandable and actionable.
- Help students plan exam preparation.
- Provide useful visual analytics without overwhelming the user.
- Provide a strong foundation for future AI-powered academic assistance.
- Deliver a responsive experience across desktop, tablet and mobile.

6. Non-Goals

- Do not replace an official college ERP in the initial release.
- Do not scrape private college systems or bypass authentication.
- Do not invent official academic information.
- Do not make AI-generated information appear as official college information.
- Do not overload the first version with unnecessary social-network features.
- Do not add libraries or services only because they are trendy; every dependency should have a clear purpose.

7. Core Product Navigation

- Home
- Today
- Timetable
- Calendar
- Tasks
- Attendance
- Exams
- Notes
- Notifications
- Profile
- Settings

Mobile navigation: Home | Today | Calendar | Tasks | More

8. Core Features

Home — Greeting, notification count, classes today, attendance, pending tasks, upcoming exams, next-class countdown, today's schedule and upcoming academic items.

Today — Previous day / Today / Next day navigation; class timeline; today's tasks; events and reminders.

Timetable — Weekly Monday-Friday schedule with subjects, time slots, rooms, faculty, labs and free periods. Support manual creation/editing in the initial release.

Calendar — Monthly calendar containing classes, assignments, exams, events and personal tasks. Selecting a date shows its events.

Tasks — All, Today, Upcoming, Overdue and Completed views. Tasks support title, subject, due date/time, priority, category, description, attachment, reminder and completion.

Attendance — Overall attendance, subject-wise attendance, attended/total classes, target percentage, warning states and a future 'what if I miss next class?' simulator.

Exams — Upcoming exams, days remaining, date/time/room/syllabus and preparation progress. Future versions can generate study plans.

Notes — Subject folders, note editor, attachments, links and search. Future versions can add AI summarization and explanation.

Notifications — Academic, assignment, exam, attendance, system and task notifications. Support configurable reminder timing.

Profile — Student identity, program, semester, section, batch and academic year, plus account and preference settings.

Settings — Appearance, notifications, account, security, language and other configurable preferences.

9. Interconnected Product Behavior

Features must not behave like isolated pages. Data created or changed in one module should automatically affect related modules.

- If a student creates a DBMS assignment due on 25 September, it appears in Tasks and Calendar, contributes to Home's upcoming items, appears in Today on the relevant date, and triggers configured reminders.
- If a timetable class is scheduled for tomorrow, it should appear in Tomorrow/Today views and contribute to the next-class information.
- If attendance changes, overall attendance, subject attendance, warnings and relevant dashboard statistics should update.
- If an exam is added, it should appear in Exams, Calendar, Home upcoming items and relevant reminders.
- Completing a task should update task statistics and remove it from pending views while retaining it in completed history.

10. Notifications and Reminders

- Class reminder
- Assignment reminder
- Exam reminder
- Attendance warning
- Task reminder
- Daily academic summary
- System notification

Reminder timing should support configurable options such as 5 minutes before, 30 minutes before, 1 day before and custom timing. The exact delivery mechanism can be finalized in the technical specification.

11. Analytics and Visual Insights

- Attendance: subject-wise bar chart or equivalent visualization.
- Study hours: line chart over time.

- Task completion: donut/pie chart.
- Exam preparation: progress bars and/or bar chart.
- Academic performance: line chart when valid historical data exists.
- Dashboard: combine summary cards with meaningful visual analytics; avoid unnecessary decorative charts.

12. AI Features — Future/Controlled Scope

- AI Study Planner: create a study plan from available tasks, exams and available time.
- AI Doubt Assistant: explain academic topics in simple language.
- AI Notes Summary: summarize user-provided notes/documents.
- AI Exam Preparation: generate revision plans and practice questions from permitted user content.
- AI Performance Analysis: identify patterns from available student data without inventing facts.
- Personalized Recommendations: suggest practical next actions based on real application data.

AI outputs must be clearly distinguishable from official college data. The AI must never fabricate exam dates, attendance records, faculty information or other authoritative academic facts.

13. Responsive and UX Requirements

- Desktop-first professional dashboard while fully supporting mobile.
- Use reusable components and consistent spacing, typography, colors and interaction patterns.
- Provide loading, empty, success and error states.
- Support keyboard accessibility and visible focus states.
- Support light and dark themes.
- Use subtle, purposeful animations; avoid excessive motion.
- Forms must provide clear validation and understandable error messages.
- Tables and charts must remain usable on smaller screens.
- Navigation must remain predictable across all pages.

14. Data Integrity and Security Principles

- Users must only access data they are authorized to access.
- Sensitive configuration must use environment variables or secure configuration management.
- Validate input on both client and server where applicable.
- Never expose secrets in frontend code or documentation.
- Use secure authentication/session practices defined in the technical specification.
- Protect API endpoints against unauthorized access.
- Use safe database queries/ORM practices.
- Do not store unnecessary personal information.

15. Initial Data Strategy

The MVP should support seeded/demo data and manual user-entered data so development does not depend on a college's internal system. Integration with an institution should be treated as a separate future capability requiring authorization.

16. MVP Scope

- Authentication and student profile
- Home dashboard
- Today view
- Timetable
- Calendar
- Tasks
- Basic notifications/reminders
- Attendance
- Exams
- Notes
- Responsive UI
- Basic analytics/charts
- Core settings

17. Future Scope

- Authorized college/ERP integration
- Advanced AI study planner
- AI academic assistant
- AI note processing
- Advanced analytics
- Teacher/mentor roles
- Institution administration
- Multi-campus support
- Advanced automation
- Native mobile applications

18. Quality Expectations

- The application should feel like a polished SaaS product rather than a basic CRUD project.
- All major user flows must work end-to-end.
- UI should remain consistent across pages.
- Errors should be handled gracefully.
- No major feature should exist only as a visual mockup; interactive controls must perform their documented behavior.
- Before considering a phase complete, run the relevant tests and manually verify the main user journeys.

19. AI Coding-Agent Rules

- Read all project documentation before implementing a major feature.
- Treat the documentation as the source of truth, but resolve conflicts explicitly rather than silently choosing arbitrary behavior.
- Do not invent undocumented product requirements.
- Use the approved open-source libraries listed in the project's Open Source documentation where appropriate.
- Build reusable components instead of duplicating UI.
- Keep frontend, backend, database and shared types/contracts organized.
- Do not hard-code production secrets or sensitive data.
- Implement real functionality, not placeholder buttons, unless the documentation explicitly marks the feature as future scope.
- After each major phase, run tests/build/lint checks and fix errors before moving forward.
- Do not remove an existing working feature merely to make a new feature easier.
- Keep the codebase maintainable and production-oriented.

20. Acceptance Criteria

- A student can log in and reach a personalized dashboard.
- A student can view today's classes and upcoming activities.
- A student can create, edit, complete and filter tasks.
- Tasks and academic events appear correctly in related calendar/dashboard views.
- A student can manage/view timetable information.
- Attendance is visible overall and by subject.
- Exams are visible with dates and preparation progress.
- Notes can be created and organized.
- Notifications/reminders follow configured settings.
- Charts display only data that actually exists.
- The application works on desktop and mobile layouts.
- Major loading, empty and error states are implemented.
- Core user flows pass functional testing.

21. Product Success Signals

- Daily active usage
- Task completion rate
- Missed-deadline reduction
- Reminder interaction rate
- Timetable/Today view usage
- Attendance feature usage
- Exam planning usage

- Retention over time

22. Relationship With Other Documentation

This PRD defines the product-level WHAT and WHY. The SRS defines detailed software behavior. Architecture defines technical structure. UI/UX and Design System define the interface. Feature Spec defines detailed feature behavior. Database/API documents define data and service contracts. Testing and Deployment documents define delivery quality and production operations.

23. Final Implementation Principle

Build the product as one connected Student OS, not as separate CRUD pages. Every major piece of academic information should answer at least one practical student question: *What is happening? What is due? What should I do? What is coming next? Am I on track?*